

Logistic Regression for Iris Dataset

Write Python code to implement logistic regression on the attached dataset. Use all the data for training the model. Report the accuracy. Advised not to use the in-built functionality available in Scikit learn. Implement both One vs Rest and One vs One approaches for multi-label classification.

Report the following in the report:

1. Expression for the Decision Boundary
2. Accuracy
3. Confusion matrix

```
In [11]: import numpy as np
import pandas as pd

from typing import Tuple, List, Dict
from collections import Counter
```

```
In [12]: class LogisticRegressionBinary :

    def __init__(self, lr: float = 0.01, epochs: int = 5000):
        self.lr : float = lr
        self.epochs : int = epochs
        self.w : np.ndarray | None = None
        self.b : float = 0.0

    def sigmoid(self, z: np.ndarray) -> np.ndarray :
        return 1.0 / (1 + np.exp(-z))

    def fit(self, X: np.ndarray, y: np.ndarray) -> None:

        n_samples: int
        n_features: int
        n_samples, n_features = X.shape

        self.w = np.zeros(n_features)
        self.b = 0.0

        for _ in range(self.epochs):

            linear: np.ndarray = X @ self.w + self.b
            y_pred: np.ndarray = self.sigmoid(linear)

            dw: np.ndarray = (1 / n_samples) * (X.T @ (y_pred - y))
            db: float = (1 / n_samples) * np.sum(y_pred - y)

            self.w -= self.lr * dw
            self.b -= self.lr * db
```

```

def predict_prob(self, X: np.ndarray) -> np.ndarray :
    assert self.w is not None
    return self.sigmoid(X @ self.w + self.b)

def predict(self, X: np.ndarray) -> np.ndarray :
    probs: np.ndarray = self.predict_prob(X)
    return (probs >= 0.5).astype(int)

```

In [13]: **class** OneVsRest:

```

def __init__(self):
    self.models: Dict[int, LogisticRegressionBinary] = {}

def fit(self, X: np.ndarray, y: np.ndarray) -> None:

    classes: np.ndarray = np.unique(y)

    for c in classes:

        y_binary: np.ndarray = (y == c).astype(int)

        model = LogisticRegressionBinary()
        model.fit(X, y_binary)

        self.models[c] = model

def predict(self, X: np.ndarray) -> np.ndarray:

    probs: List[np.ndarray] = []

    for c in sorted(self.models.keys()):
        probs.append(self.models[c].predict_prob(X))

    prob_matrix: np.ndarray = np.vstack(probs).T

    return np.argmax(prob_matrix, axis=1)

```

In [14]: **class** OneVsOne:

```

def __init__(self):
    self.models: Dict[Tuple[int, int], LogisticRegressionBinary] = {}

def fit(self, X: np.ndarray, y: np.ndarray) -> None:

    classes: np.ndarray = np.unique(y)

    for i in range(len(classes)):
        for j in range(i + 1, len(classes)):

            c1 = classes[i]
            c2 = classes[j]

            mask = (y == c1) | (y == c2)

```

```

        X_pair = X[mask]
        y_pair = y[mask]

        y_binary = (y_pair == c1).astype(int)

        model = LogisticRegressionBinary()
        model.fit(X_pair, y_binary)

        self.models[(c1, c2)] = model

    def predict(self, X: np.ndarray) -> np.ndarray:

        predictions: List[int] = []

        for x in X:

            votes: List[int] = []

            for (c1, c2), model in self.models.items():

                pred = model.predict(x.reshape(1, -1))[0]

                if pred == 1:
                    votes.append(c1)
                else:
                    votes.append(c2)

            predictions.append(Counter(votes).most_common(1)[0][0])

        return np.array(predictions)

```

```

In [15]: def accuracy(y_true: np.ndarray, y_pred: np.ndarray) -> float:
        return np.mean(y_true == y_pred)

def confusion_matrix(y_true: np.ndarray, y_pred: np.ndarray) -> np.ndarray:

    classes = np.unique(y_true)
    n = len(classes)

    matrix = np.zeros((n, n), dtype=int)

    for t, p in zip(y_true, y_pred):
        matrix[t][p] += 1

    return matrix

```

```

In [16]: df: pd.DataFrame = pd.read_csv("iris.csv")

X: np.ndarray = df.iloc[:, :-1].values
y_labels: np.ndarray = df.iloc[:, -1].values

# convert string labels to integers

```

```
classes = {label: idx for idx, label in enumerate(np.unique(y_labels))}
y: np.ndarray = np.array([classes[label] for label in y_labels])
```

```
In [17]: ovr = OneVsRest()
         ovr.fit(X, y)

         pred_ovr = ovr.predict(X)

         print("OvR Accuracy:", accuracy(y, pred_ovr))
         print("OvR Confusion Matrix:")
         print(confusion_matrix(y, pred_ovr))
```

OvR Accuracy: 0.9733333333333334

OvR Confusion Matrix:

```
[[50  0  0]
 [ 0 46  4]
 [ 0  0 50]]
```

```
In [18]: ovo = OneVsOne()
         ovo.fit(X, y)

         pred_ovo = ovo.predict(X)

         print("OvO Accuracy:", accuracy(y, pred_ovo))
         print("OvO Confusion Matrix:")
         print(confusion_matrix(y, pred_ovo))
```

OvO Accuracy: 0.98

OvO Confusion Matrix:

```
[[50  0  0]
 [ 0 47  3]
 [ 0  0 50]]
```